What Is the Playwright Framework and is it Build on Selenium?

Playwright is a popular web testing tool that allows you to **test modern web apps with ease**.

Your first question might be Is Playwright built on Selenium? No, Playwright is not built on Selenium.

How did Playwright Start

Playwright started as a fork of Puppeteer, Google's Node library for browser automation, but has diverged with a different philosophy and feature set geared towards end-to-end testing. While the Selenium WebDriver is a W3C standard with broad adoption, Playwright takes a fresh approach aimed at making modern web testing simpler and more capable.

Because of this it has certain features that make it stand out compared to other tools based on Selenium.

What are some standout features of Playwright?

One of the key features is its ability to **eliminate flaky tests** by auto wait for elements to be actionable prior to performing actions. It allows developers to write scripts that interact with web pages and simulate real user behavior, like clicking buttons and filling out forms.

Other notable features are the **support for iframes, built-in reporters like HTML reports, and shadow dom**.

Playwright has the ability to **test web applications across different browser versions and platforms**, including desktop and mobile devices. It supports headless testing, which means that tests can be run without the need for a graphical user interface. This makes it possible to run tests in a CI/CD pipeline or on a remote server, which is ideal for testing applications that need to run on different operating systems or devices.

Playwright also provides a **high-level API** for interacting with web pages, which makes it developer friendly to write automated tests that are both concise and expressive.

Other features mentioned on my podcast that testers say they like:

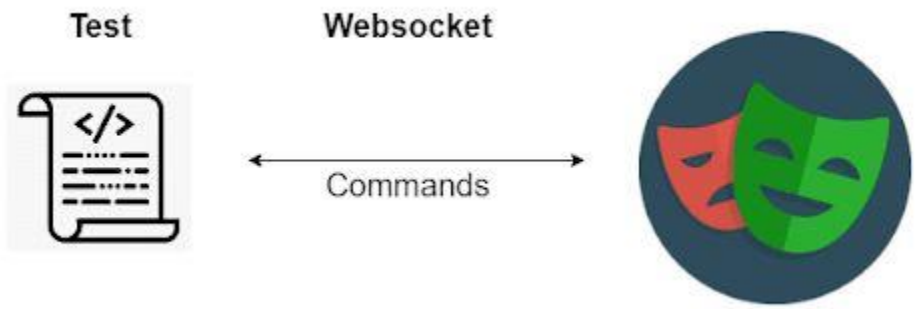
- Auto-waiting for elements to be ready before executing actions
- Multi-page scenarios and frames with the Browser Context concept
- Cross-browser support with a single API
- Mobile emulation with device viewport, geolocation and permissions

- Network interception, mocking and request/response awaiting
- Shadow DOM support
- First-class TypeScript support
- Visual comparisons, with built-in tooling
- Parallelization and sharding from a single browser instance
- Integrations with popular test runners like Jest, Mocha
- Detailed, actionable debugging with tracing and source-maps
- Broad CI integration with Docker images, GitHub Actions

Playwright Architecture

We have mentioned some of the reasons why Playwright is great at the functional and implementation level, but what happens internally? What makes test execution more stable and faster?

Playwright c**ommunicates all requests over a single WebSocket connection** that remains in place until the test execution is complete. This reduces the sources of errors and allows commands to be sent quickly over a single connection.



How does Playwright improve ergonomics compared to Puppeteer?

While Playwright started as a Puppeteer fork, it adds compelling improvements for test automation:

- Cross-browser support with a consistent API
- Auto-waiting and timeout configurations for more reliable tests
- A powerful locator engine and a Locator API for resilient selectors
- Broad mobile emulation and device support

- Parallelization with browser contexts
- In-built test runner, reporters, and assertions
- Tracing for easy troubleshooting

How Cross-browser Web Automation Works With Playwright

Cross-browser testing is an essential part of web application testing. Because different browsers render pages differently, it's important to test your application across multiple browsers to ensure that it works correctly for all users.

Playwright makes testing easy by providing a unified API for interacting with web browsers. This means that developers can write tests once and run them across multiple browsers, without the need to write separate code for each browser.

Playwright is a **cross-browser web automation framework that supports all major web browsers**, including Chromium, Firefox, and WebKit. It provides a range of features that are designed to make cross-browser testing easier, including automatic browser installation, browser context management, and browser configuration.

Playwright works by launching a browser instance in headless mode, which means that the browser window is not visible on the screen. It then navigates to the desired URL and performs the specified actions, such as clicking a button or filling out a form.

Playwright also supports running tests in non-headless mode, which allows you to see the browser window as the tests run. This can be helpful for debugging and troubleshooting.

How does Playwright make it easy to get started with browser automation?

Playwright offers an easy API to automate Chromium, Firefox and WebKit with a single library. The setup is straightforward – you can install Playwright with a single command and get started with the in-built test runner. The documentation site provides clear guides, examples, and an API reference. You can easily integrate Playwright tests into CI/CD pipelines.

How To Install Playwright via Command Line

Installing Playwright via the command line is straightforward. Follow these steps:

1. Install Node.js on your machine if you haven't already. Download it from the official website.
2. Open a terminal or command prompt, then run the below command to install Playwright:

npm init playwright@latest tutorial

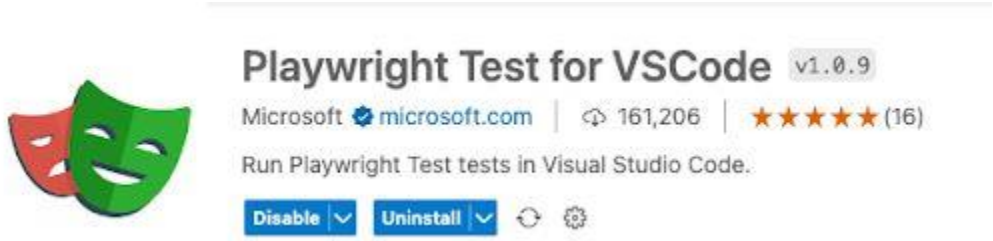
After performing this operation, you will get several prompts showing you how you want to configure the new project. These can be selected using the arrow keys or the keyboard. Once you have selected all the options, the project will be initialized.

That means it will install the latest version of Playwright and all its dependencies. You're now ready to start using Playwright in your project!

How To Install Playwright via Visual Studio Code

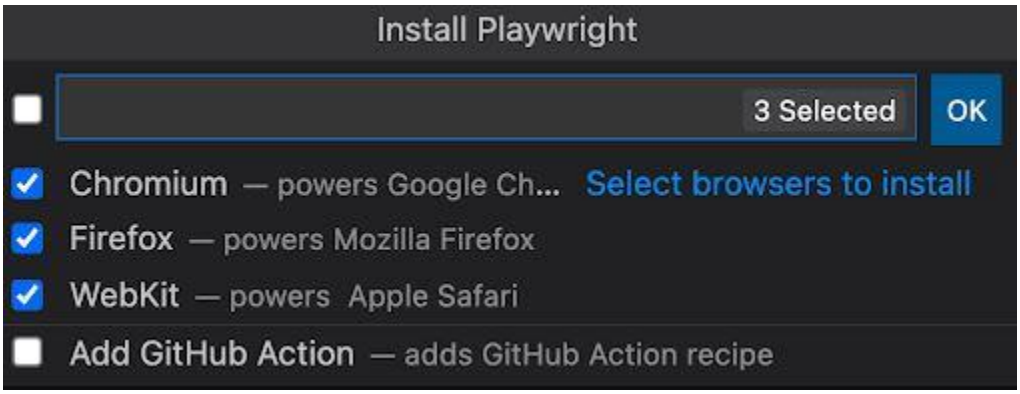
If you prefer to use Visual Studio Code as your code editor, you can also **install Playwright directly from within the editor**.

In VS-Code, open the extensions and search for Playwright. Make sure you choose the one that is verified by Microsoft.



Once the extension is installed, you can begin installing Playwright in your project using the command bar and then type 'install playwright'.

You can then choose whether to run the tests on Chromium, WebKit, or Firefox, and you can also decide to add GitHub actions as a CI environment.



How Is the Project Structure After Installation?

After installing Playwright, your project structure will look something like this:

```
.
├── node_modules
├── package-lock.json
├── package.json
├── playwright.config.ts
├── tests
└── tests-examples
```

- package.json file contains information about your project, including the version of Playwright you're using. Unless you are already in a project that contains this file, Playwright will be added to your package.json
- playwright.config.ts file is the place where you can configure the playwright library, add or remove all the major browsers you want to test, and more. You can also configure timeouts, parallelism, retires, and reporters, just a brief example! Playwright supports tons of customization and adjustment to meet your test suite's requirements.
- The tests folder contains an example spec file. That is generated when initializing a new project. This is ideal for understanding the syntax and a good starting point to create scenarios.

Run the First Playwright Test Case With the Playwright Runner

Via command line

Once you have Playwright installed, you can start writing test cases.

Since Playwright is an automation framework it also comes with its own playwright test runner that you can use to run your tests from the command line.

1. Create a new TypeScript file in your tests directory and write your Playwright test script.
2. Save the file with a .spec.ts extension.

3. Open your terminal, then navigate to the directory where your project is located.
4. Type the below command:

npx playwright test

This will launch the Playwright test runner and run playwright tests in all the browsers and generates an HTML report of the test results.

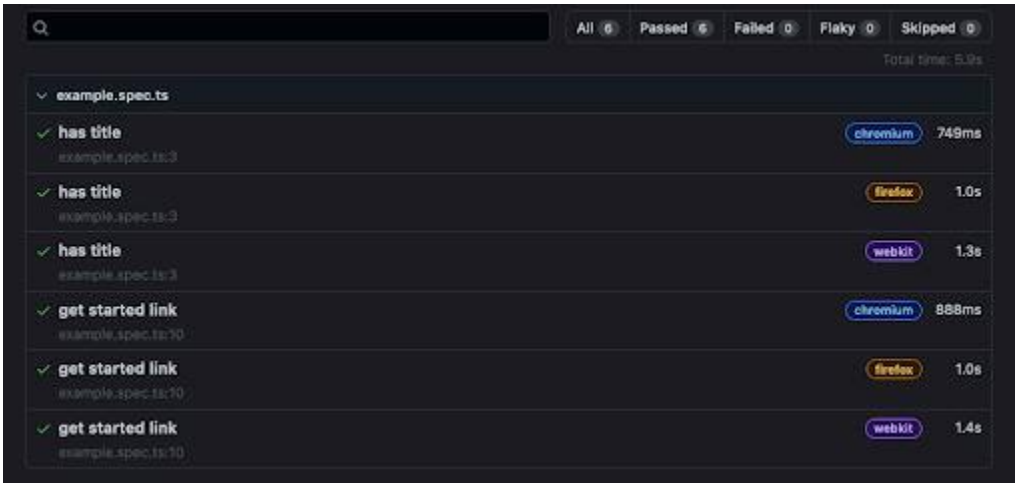
```
npx playwright test

Running 6 tests using 5 workers
  6 passed (5.9s)

To open last HTML report run:

  npx playwright show-report
```

We have 6 passing tests using 5 workers being used. This means 5 different processes running in parallel to execute our tests.

If you want to view the tests you just ran, open the example.spec.ts file. The test cases are identified by the syntax “test.describe”, and at the top of the file, you can see the required packages, the steps performed before each test, and some list data used in the tests.

To open the HTML report execute the following command:

npx playwright show-report



You can specify a different browser using the –browser flag:

npx playwright test –browser=firefox

You can also specify the path to the test file or directory using the –test flag:

npx playwright test –test=path/to/test.spec.ts

You can pass additional options to the test runner using the –config flag. For example, to run tests in headless mode, you can use the following command:

npx playwright test –config=headless=true

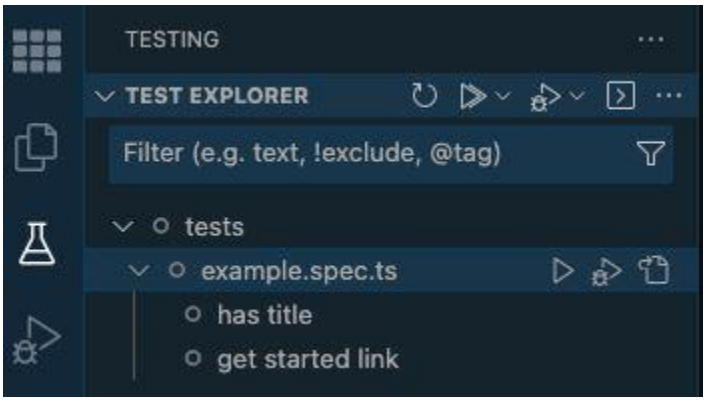
Via Visual Studio Code

To run your first playwright test with the VS-Code plugin, you can follow these steps:

Create a new spec file under the tests folder and write your test automation code using the test function provided by Playwright Test. For example:

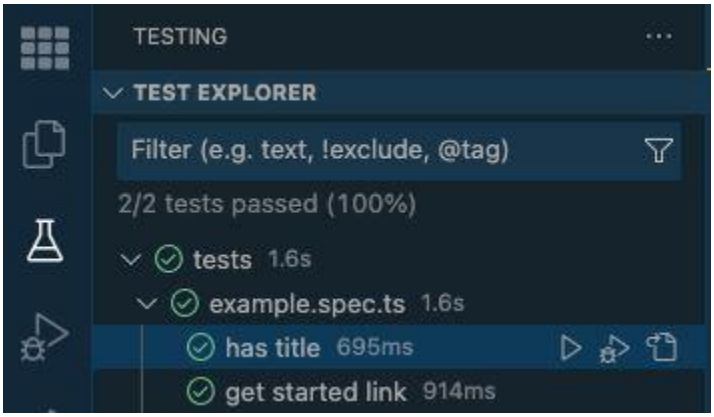
```
tests > example.spec.ts > ...
1   import { test, expect } from '@playwright/test';
2
3   test('has title', async ({ page }) => {
4     await page.goto('https://playwright.dev/');
5
6     // Expect a title "to contain" a substring.
7     await expect(page).toHaveTitle(/Playwright/);
8   });
9
10  test('get started link', async ({ page }) => {
11    await page.goto('https://playwright.dev/');
12
13    // Click the get started link.
14    await page.getByRole('link', { name: 'Get started' }).click();
15
16    // Expects the URL to contain intro.
17    await expect(page).toHaveURL(/.*intro/);
18  });
```

Next, open the “Test” view in VS-Code by clicking on the “Test” icon in the activity bar on the side of the window.



In the “Test” view, you should see your test case listed under your test file. You can run your tests by clicking on the “Run Test” button or by right-clicking on an individual test and selecting “Run Test”.

This will run your automated tests using the Playwright Test runner and display the results in the “Test” view within VS-Code.



How To Handle Multiple Browser Tabs With Playwright?

Testing multiple browser tabs is a common scenario in web automation. With Playwright, you can easily create and interact with multiple tabs to perform complex tasks.

In this section, we’ll explore how to test multiple browser tabs with Playwright.

First, let's understand how Playwright handles multiple tabs. Playwright also supports multiple browser contexts and multiple pages, which makes it easy to test complex web applications. You can also handle multiple browser tabs with Playwright, making it easy to create test scenarios.

Each browser context in Playwright can have one or more pages, and each page represents a tab. You can create a new page by calling the newPage() method on a browser context. By default, the first page is created automatically when the context is created.

To test multiple tabs, we will create two pages and navigate them to different URLs. Here's an example:

```
import { test, expect } from ‘@playwright/test'


test(‘Open new tab', async ({ page }) => {
  // Open the web application
  await page.goto(‘https://the-internet.herokuapp.com/windows')


  // Emitted when the page opens a new tab or window
  const newPagePromise = page.waitForEvent(‘popup');


  // Click the link locator
  await page.getByRole(‘link', { name: ‘Click Here' }).click();


  // Assign the new page promise
  const newPage = await newPagePromise;


  // Validate the title of the new page
  await expect(newPage).toHaveTitle(/New Window/);
});
```

This code waits for a new page to be opened before clicking on an element with name ‘Click Here'. Once the page is opened, it waits for it to load and then validates its title.

Switching Tabs

You can also switch between pages using the bringToFront() method. This method activates the page and brings it to the front of the browser window. Here's an example:

```
await page.bringToFront();
```

In this example, we switch to page by calling the bringToFront() method.

Finally, you can close a page using the close() method. Here's an example:

```
await newPage.close();
```

In this example, we close newPage by calling the close() method.

Overall, testing multiple tabs with Playwright is a powerful feature that allows you to perform complex tasks that involve multiple web pages.

With **Playwright's advanced features and the Playwright API**, you can easily create and interact with multiple tabs in your test automation.

Which Playwright Tests Do You Need To Check Out?

Playwright comes with a set of built-in tests that you can use as examples or starting points for your own tests. These tests cover a range of scenarios, including navigation, form filling, and UI interactions.

To run the example, clone the Playwright repository from GitHub, navigate to the examples directory, and execute the following command:

```
npm run test
```

This will run all the tests and generate a full HTML report of your tests. This allows you to filter the report by browsers, failed tests, passed tests, skipped tests, and flaky tests.

The Playwright documentation provides a variety of tests that developers can use to get started with the framework.

These include tests for basic interactions, page navigation, and file uploads. Developers should also check out the Playwright test runner documentation to understand how to run tests effectively.

Which Programming Languages Are Supported?

Playwright supports multiple programming languages, including:

- JavaScript/TypeScript
- Python
- Java
- .NET

This means that you can write tests in your preferred language and still use the same powerful features of Playwright.

The **choice of programming language will depend on your team's expertise and the tools and frameworks** you are already using.

For example, if you are using Node.js for your web development, then JavaScript or TypeScript would be a natural choice for writing Playwright tests.

Similarly, if you are using Python for your data analysis or machine learning tasks, then you may prefer to use Python for your test automation as well.

Playwright also provides detailed documentation and examples for each supported language, making it easy to get started with Playwright regardless of your programming background.

What else?

You can **run parallel tests with multi-browser support** using Playwright out of the box. You can also perform API testing and handle network requests with Playwright.

Playwright **provides tools support for writing tests**, including the Playwright inspector, allowing you to step over each Playwright API call to inspect the page. You can also use the trace viewer to view the performance of your tests.

It also **provides HTML reports**, making it easy to view the results of your tests.

Playwright **trace contains test execution screencast, live DOM snapshots, action explorer, test source**, and many more.